

Homework 1 — Tokenization and Word Vectors

1. Environment

```
python >= 3.9
numpy, matplotlib
tokenizers, gensim, scipy
jupyter
```

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
```

Everything runs on CPU. The reference implementation takes about fifteen minutes end to end on two cores, and five to eight minutes on a current laptop. The Word2Vec cells in Part 2 are the slow ones.

2. Corpus

```
python prepare_corpus.py
```

File	Size	Used by
data/corpus_train.txt	20M words	Part 1-6, Part 2
data/corpus_valid.txt	500K words	token counts, evaluation
data/corpus_small.txt	5 MB	Part 1, your own implementation

Compare the generated `data/checksums.txt` against the published SHA-256 values. Contact the TA if they differ.

Two alternative sources:

```
python prepare_corpus.py --source github --train-words 5500000 # Hugging Face blocked
python prepare_corpus.py --input <path> # local text file
```

`data_smoke/` holds a corpus of about 45,000 words. Point the notebook at it with `HW1_DATA=data_smoke` to check that everything runs before you commit to the full corpus.

3. What you implement

File	What you implement	Points
bpe.py	count_words, train_bpe, build_vocab, tokenize, encode, decode	35
hw1.ipynb Part 1	cells 1-3, 1-4, 1-6	10
hw1.ipynb Part 2	cells 2-4, 2-5, 2-5b, 2-6, 2-7	20
report.pdf	the seven questions in report_template.docx	35

Each body you have to write is marked with `TODO: implement this` followed by `raise NotImplementedError`. Delete both lines and write your code there. Do not change the function signatures, the cells that are already filled in, or `hw1_helpers.py`; the self-check and the grading assume them.

Read the questions in `report_template.docx` before you start. They are where most of the learning is; the code exists to produce the evidence for them.

4. Rules

In `bpe.py` you may use the standard library only. Do not import `tokenizers`, `transformers`, or `numpy` in that file. This is the one restriction that matters, because `bpe.py` is the algorithm the homework is about.

Everywhere else — the notebook, the figures, the report — `numpy`, `matplotlib`, `tokenizers` and `gensim` are expected. Part 1-4 exists precisely to compare your implementation against the library one.

5. Self-check

```
python test_bpe_public.py
```

Six checks run in a second. They reproduce the lecture example on slides 15-21 and the round trip. They report `TODO` for anything you have not implemented yet, so they are useful from the first function onwards.

`test_bpe_public.py` is a debugging aid, **not the grader**. A further set of hidden tests covers tie breaking, the stopping condition, the vocabulary order and unknown characters. Passing every public check does not mean full marks: the tables and the written answers count for as much as the code.

Paste the full output into section 2 of your report.

6. Conventions you must follow

These are fixed so that every submission can be compared against the same reference. Getting them wrong changes every number in your report.

No word-boundary marker. The base vocabulary is the set of characters in the training corpus, and `hugs` splits into `h+u+g+s`, as on slide 15. Adding an end-of-word or start-of-word marker changes the second merge and every number after it.

Merge selection. Take the most frequent pair; break ties alphabetically; never merge a pair whose frequency is below 2.

Encoding. Split on whitespace and replay the merges on each word independently, merging the highest-ranked adjacent pair first. Insert `<sp>` between words; it takes no part in training and exists only to restore whitespace.

Unknown characters. A character absent from the training corpus encodes as `<unk>`. Do not raise an exception.

Token ids. `build_vocab` orders tokens as `<unk>`, `<sp>`, base characters in alphabetical order, then merged tokens in the order they were learned. That order defines the ids.

Performance. On the 5 MB slice, roughly 50,000 word types, 1,000 merges must finish within a few minutes. Keep an index from each pair to the words that contain it and update only the affected words. The reference implementation takes about three seconds.

7. Submission

Submit a single archive named `HW1_<student id>_<name>.zip` containing:

```
HW1_<student id>_<name>/
├─ bpe.py           your implementation
├─ hw1.ipynb        the notebook with its outputs
├─ results/         the CSV files and the PNG your cells produced
└─ report.pdf       report_template.docx filled in, at most 6 pages
```